



Lecture 13

Microservice Workflow

Sharif University of Technology
Winter 2026



Learning Objectives

Explain ACID properties and why they matter in database design

Describe the challenges of maintaining atomicity across microservices

Understand the Two-Phase Commit protocol and its limitations

Design workflows using the Saga pattern (orchestration & choreography)

Implement compensating transactions for rollback scenarios

Choose between orchestrated vs choreographed sagas for a given problem



Agenda



Part 1

Database Transactions

- What is a transaction?
- ACID properties
- Atomicity in detail

Part 2

Distributed Systems Challenges

- Cross-service transactions
- Two-Phase Commit
- Failure scenarios
- Why 2PC falls short

Part 3

Sagas Pattern

- Saga motivation
- Failure modes
- Orchestration
- Choreography



Part I

Database Transactions

Understanding the foundation before we break it apart

What is a Database Transaction?



- A transaction groups one or more operations into a single logical unit
- Either ALL operations succeed (commit), or ALL fail (rollback)
- Prevents partial updates that leave data in an inconsistent state
- The database guarantees the outcome — no matter what happens

BEGIN

INSERT ...

UPDATE ...

COMMIT ✓



Real-world analogy

A bank transfer: debit \$100 from Account A AND credit \$100 to Account B. Both must happen. If Account B doesn't exist, Account A must NOT be debited either.

ACID Properties



The four guarantees every reliable database must provide

A Atomicity

All operations in the transaction complete, or none of them do.

C Consistency

The database is always left in a valid state after a transaction.

I Isolation

Concurrent transactions operate independently without interference.

D Durability

Once committed, data persists even through crashes or power failures.

Atomicity: All or Nothing



- Atomicity is the 'all-or-nothing' guarantee
- If any single operation in a transaction fails, the entire transaction is rolled back
- No partial updates — the database looks as if the transaction never happened
- Achieved via write-ahead logging (WAL) in most relational databases



```
1 from sqlalchemy.orm import Session
2
3 def transfer_money(db: Session, from_id: int,
4                     to_id: int, amount: float):
5     try:
6         sender = db.query(Account).get(from_id)
7         receiver = db.query(Account).get(to_id)
8
9         sender.balance -= amount # Step 1
10        receiver.balance += amount # Step 2
11
12        db.commit()
13    except Exception:
14        db.rollback()
15        raise
```

⚠ The microservices problem: each service has its OWN database. A single ACID transaction cannot span across services. We lose atomicity the moment we split our data.



Isolation: Concurrency Without Chaos



Read Uncommitted

⚠ Rarely used

Your transaction can read data that another transaction has changed but not yet saved (committed). If that other transaction then cancels, you have read data that never officially existed.

💬 You see a price of €5 — but it was being edited and got rolled back to €10. You read a ghost value.

Read Committed

✓ Most used

You only read data that has been fully saved by other transactions. You will never read a 'ghost' value. However, if you read the same row twice, it might change between your two reads.

💬 You read a balance of €100. Another transaction deducts €20 and commits. You read again: now €80. Same transaction, two different answers.

Repeatable Read

→ Stronger guarantee

Any row you read once will return the same value if you read it again within the same transaction — even if another transaction changes it in between.

💬 You check a stock quantity: 50 units. Mid-transaction, someone sells 10. You check again: still 50. Your view is frozen.

Serializable

⚡ Strongest / slowest

Transactions behave as if they ran one after another, never at the same time. Complete protection — but the database uses more locking, which slows things down.

💬 Two users book the last seat on a flight simultaneously. Serializable ensures only one succeeds — never a double-booking.

Durability: Surviving Failures



- Once a transaction commits, the data is written to persistent storage
- Achieved through Write-Ahead Logging (WAL): changes are logged BEFORE applied
- If the database crashes mid-write, the WAL allows replay on restart
- PostgreSQL, MySQL, Oracle all use WAL (or equivalent) for durability

1. Transaction starts

2. Write to WAL log

3. Flush WAL to disk

4. Apply to data file

5. Commit confirmed ✓

In microservices: each service has its own durable store. Durability is still guaranteed per-service — the challenge is coordinating across services, not within one.

ACID in a Single Microservice — Still Works!



- A single microservice CAN use ACID transactions against its own database
- The scope is limited to state changes within that one service
- Problem starts when ONE business operation spans MULTIPLE services
- Classic example: onboarding a new customer (Customer service + Enrollment service)

✓ Works great! The issue arises when we split this across separate services...

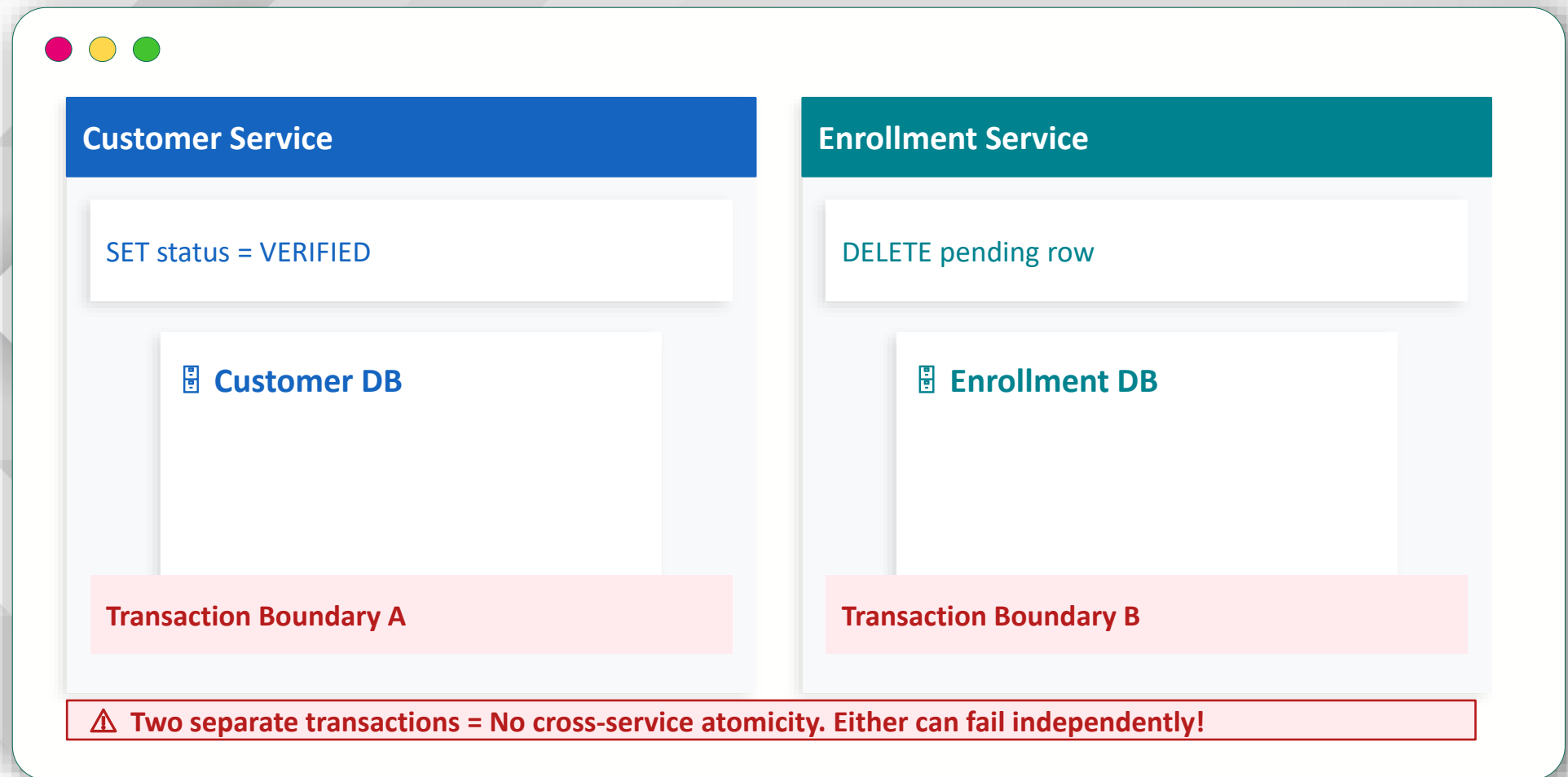
Single Service — Single DB

```
UPDATE customers  
SET status='VERIFIED'
```

```
DELETE FROM pending_enrollments  
WHERE cust_id=2346
```

✓ Single ACID boundary — atomicity guaranteed

The Problem: Split Across Two Services





Part II

Distributed Transactions

The obvious solution — and why it often makes things worse

The Obvious (Flawed) Solution



Spanning a single transaction across multiple services

- Idea: extend the transaction boundary to include all participating services
- If all services commit together, we get atomicity across the whole operation
- The most common algorithm for this is called the **Two-Phase Commit (2PC)**
- Many databases and middleware support 2PC natively
- Sounds great in **theory**... but the **practice** is surprisingly painful



Distributed transactions may not solve your problems and may bring even more confusion to your system.

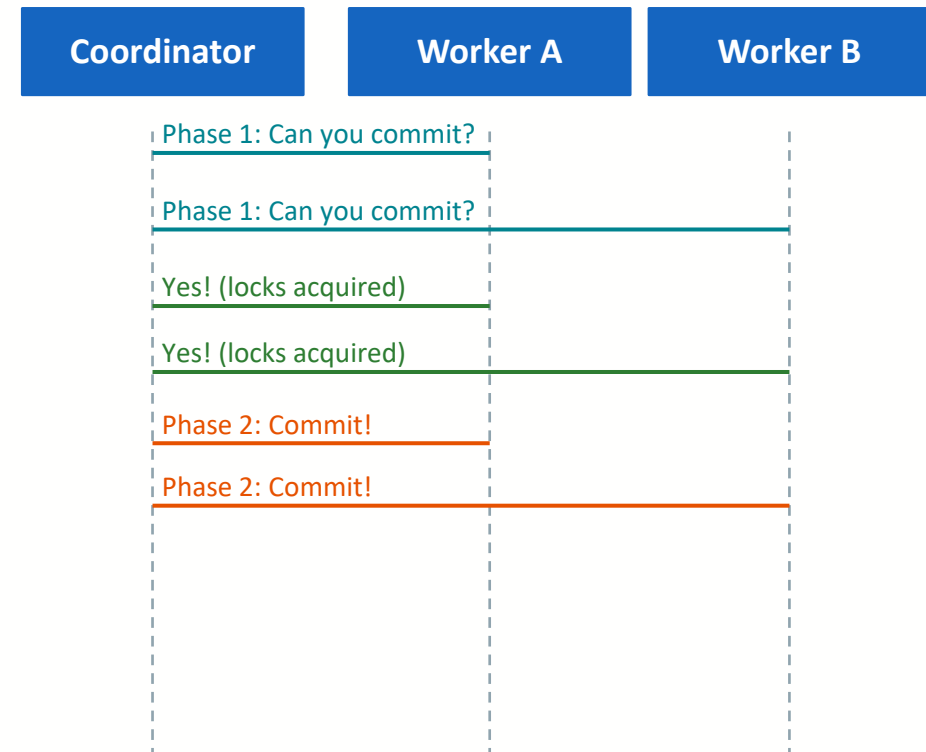
— Sam Newman
Building Microservices

Two-Phase Commit (2PC): The Algorithm



Phase 1: Vote → Phase 2: Commit

- **Phase 1 — Voting Phase:**
 - Coordinator asks all workers: 'Can you commit this change?'
 - Workers lock resources and reply Yes or No
- **Phase 2 — Commit Phase:**
 - If ALL said Yes → Coordinator sends Commit to everyone
 - If ANY said No → Coordinator sends Rollback to everyone
 - Workers apply changes and release locks



2PC: The Hidden Locking Cost



Resources are locked between Phase 1 and Phase 2

- Between voting yes and actually committing, workers MUST lock the affected rows/resources
- This prevents other transactions from modifying the same data
- Lock duration = time from vote to commit confirmation (network round trips!)
- In distributed systems: high latency = longer locks = more contention
- More participants = exponentially more lock coordination complexity

The Deadlock Trap:

Service A holds Lock 1, waits for Lock 2. Service B holds Lock 2, waits for Lock 1. Neither can proceed — the system hangs. Managing deadlocks in a single-process system is hard; across multiple networked services it becomes a nightmare.

2PC Failure Scenarios



Worker fails after voting Yes

Worker voted to commit, then crashed. Coordinator sent commit but worker never received it. Lock is held indefinitely — operator intervention required.

Coordinator fails mid-commit

Phase 2 commit sent to Worker A but not B. Worker A committed, Worker B is stuck holding locks, awaiting a commit that may never come. Inconsistent state.

Network partition during voting

Coordinator can't hear back from Worker B. Is it a 'No' or just a timeout? 2PC must assume failure → abort, but the abort message may also be lost.

Slow worker holds system hostage

One slow service delays the entire commit. All other workers hold their locks while waiting. High latency = high contention = degraded throughput.

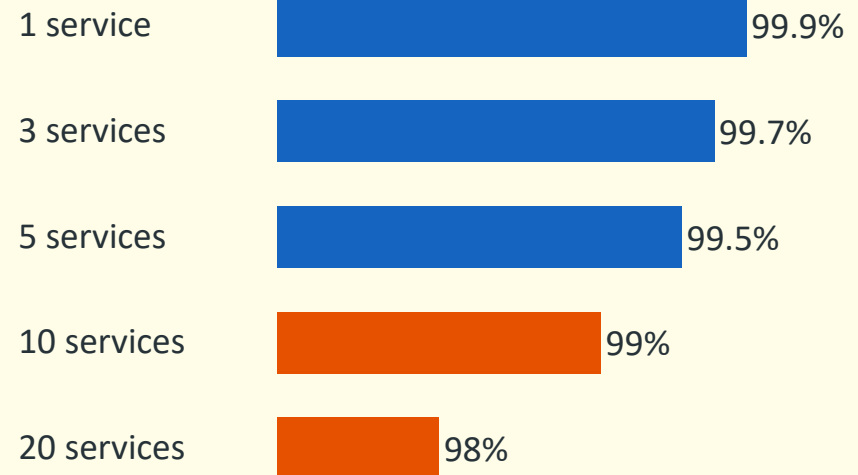
Key insight: the more nodes + the higher the latency → the wider the failure window. 2PC does not scale well.

Why 2PC Fails at Scale: Real-World Evidence



- Google Spanner: Distributed Transactions at Global Scale
 - Supports ACID transactions across geographically distributed data centers
 - Relies on specialized infrastructure:
 - Highly reliable global networking
 - Synchronously replicated data across regions
- Netflix, Amazon, Uber moved away from distributed transactions
 - They embrace eventual consistency and compensating logic instead

Availability with 2PC



(combined with 99.9% per-service uptime)

Distributed Transactions — Just Say No



- **Sam Newman's advice: avoid distributed transactions across microservices**
- Distributed transactions add complexity, reduce availability, and limit scalability
- First escape route: DON'T split the data if you need true atomicity
 - Keep tightly-coupled data in one service's database
 - Decompose elsewhere; come back to this boundary later
- **Second escape route: the SAGA pattern**
 - Model multi-step processes as sequences of local transactions
 - Handle failure explicitly with compensating transactions

2PC vs Sagas

	2PC	Sagas
Atomicity	Strong	Eventual
Locking	Long-lived	None
Failure handling	Operator-dependent	Compensations
Scalability	Poor	Good
Complexity	Protocol	Business logic



Part III

The Saga Pattern

Coordinating multi-step workflows without distributed locking

What is a Saga?



Long-lived transactions broken into short local ones

- Originally proposed by Garcia-Molina & Salem (1987) for Long-Lived Transactions (LLTs)
- An LLT is an operation that takes minutes, hours, or days — locking a database for that long is catastrophic
- **Core idea: break the LLT into a sequence of independent local transactions**
- Each step executes independently in its own ACID transaction
- The entire saga spans across multiple services without any distributed locking
- If a step fails, execute **compensating transactions** to undo previous steps

LLT → Saga decomposition

LLT: One big transaction (minutes/hours)

↓ *decompose*

T₁ (local) ✓ short lock, own DB

T₂ (local) ✓ short lock, own DB

T₃ (local) ✓ short lock, own DB

Important: Sagas Are NOT ACID Transactions



⚠ Sagas do NOT give you atomicity at the saga level

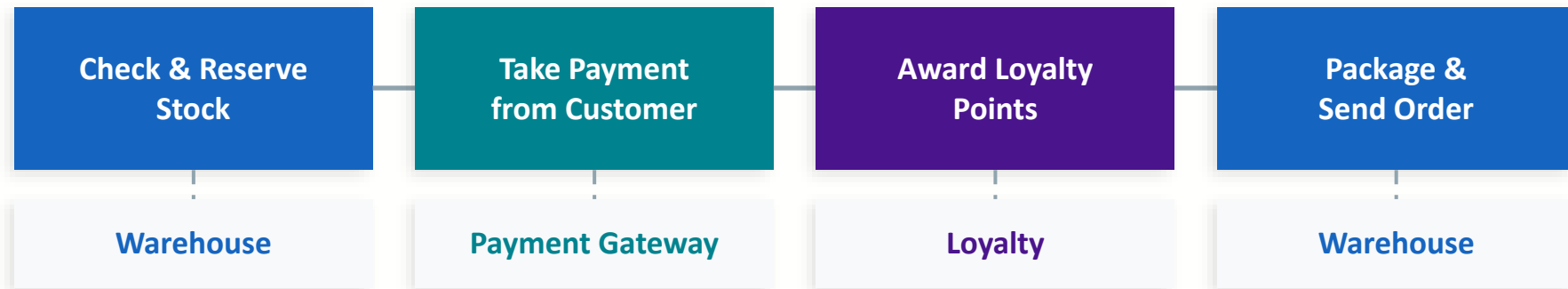
Each individual step may commit before the next step even starts. Intermediate states ARE visible. You trade strong consistency for availability and scalability.

Property	ACID Transaction	Saga
Atomicity	Whole tx all-or-nothing	Each step only; rollback via compensation
Consistency	Enforced at commit	Eventually consistent
Isolation	Other txs can't see interim state	Interim state IS visible
Durability	Guaranteed at commit	Each step is durable individually
Failure recovery	Automatic rollback	Explicit compensating transactions
Scope	Single database	Multiple services/databases

Saga in Action: Order Fulfillment



Each step handled by a different service



- Each step is a local ACID transaction — fast, no cross-service locking
- Each step is handled by the appropriate microservice
- Steps are sequenced: next step starts only after the previous one completes
- The saga tracks overall state — knows which steps have committed
- If any step fails, compensating transactions undo the preceding steps

Saga Failure Modes



Backward Recovery (Fail Backward)

- Roll back the saga from the point of failure
- Execute compensating transactions for each already-committed step
- Goal: return the system as close as possible to its pre-saga state
- Example: Payment taken but stock unavailable → refund the payment
- Think of it as: 'undo what we did, move on'

Forward Recovery (Fail Forward)

- Don't roll back — retry the failed step and continue
- Requires persisting saga state so retries are safe
- Steps must be idempotent (retrying gives same result)
- Example: Package dispatch failed → retry the next day
- Think of it as: 'push through, eventually finish'

A real saga often mixes both: fail backward on critical failures, fail forward on transient issues.

Sagas Handle Business Failures, Not Technical Failures



- **Business failure: The operation was valid, but the business rule was violated**
 - Customer has insufficient funds → saga should handle this via compensation
 - Item out of stock → saga should roll back the order
- **Technical failure: The infrastructure itself broke down**
 - Payment gateway timed out (500 error) → NOT a saga concern
 - Database connection dropped → NOT a saga concern
 - Technical failures need separate reliability patterns (circuit breakers, retries)

Is this a saga problem?

Insufficient funds	YES — business failure
Item out of stock	YES — business failure
HTTP 500 from service	NO — technical failure
DB connection timeout	NO — technical failure
Max retries exceeded	NO — technical failure

Compensating Transactions: Semantic Rollbacks



You can't un-ring a bell — but you can send an apology

- **A compensating transaction UNDOES a previously committed transaction**
- Unlike a normal rollback (which erases the change), compensation CREATES a new transaction
- Some compensations are imperfect — you cannot un-send an email, but you CAN send a cancellation email
- These are called semantic rollbacks — they restore business meaning, not bit-for-bit state
- Persistence of rollback info is valid and often important (audit trail, analytics)

Rule of thumb: for every forward action in your saga, define a compensating action before you implement the saga.



```
1  async def reserve_stock(order_id, item_id):
2      # Forward action
3      reservation = Reservation(order_id=order_id)
4      db.add(reservation)
5      db.commit()
6
7  async def cancel_stock_reservation(order_id):
8      # Compensating action
9      db.query(Reservation)\
10         .filter_by(order_id=order_id)\
11         .delete()
12      db.commit()
13
14  async def charge_payment(order_id, amount):
15      # Forward
16      charge = Payment.charge(order_id, amount)
17      return charge.id
18
19  async def refund_payment(charge_id):
20      # Compensating
21      Payment.refund(charge_id)
```

Rollback Walkthrough: Order Fulfillment Failure



Step	Forward Action	Status	Compensating Action
1	Reserve stock (Warehouse)	✓ Done	Remove stock reservation
2	Charge payment (Gateway)	✓ Done	Issue refund
3	Award loyalty points (Loyalty)	✓ Done	Deduct loyalty points
4	Package & send (Warehouse)	✗ FAILED	N/A — never completed

↑ Failure at Step 4 triggers compensating transactions for Steps 3, 2, 1 (in reverse order)

- Compensations execute in REVERSE order (3 → 2 → 1)
- Each compensation is its own local transaction — no distributed locking
- If compensation fails, retry it (it must be idempotent)
- Keep audit records — they are valuable business data

Optimization: Reorder Steps to Minimize Rollbacks



Put the most likely failures early, expensive-to-rollback steps late

- **Tip: move steps that are hard to compensate LATER in the saga**
- Move steps most likely to fail EARLIER — **fail fast**, avoid deep compensation chains
 - Example: move 'Award Loyalty Points' AFTER order dispatch
 - If packaging fails, no loyalty points were awarded, so no compensation needed for that step
- Same principle as 'fail fast' in code — detect issues before side effects accumulate

After reordering: Award Points is now step 4 — if Package fails, no points were given, nothing to undo!

Before (suboptimal)

1. Reserve Stock

2. Charge Payment

3. Award Points X

4. Package & Send

→ If step 4 fails, must undo steps 1-3 including points

Quick Check



Scenario:

You're building an e-commerce system with two microservices:

- Order Service (PostgreSQL) — creates an order record
- Inventory Service (PostgreSQL) — reserves stock

Question:

A customer places an order. The Order Service creates the record, but then the Inventory Service crashes before reserving stock. What are the possible outcomes? How would you use 2PC vs a Saga to handle this?

Quick Check: Answer & Discussion



With 2PC:

- Coordinator asks both services: 'Can you commit?'
- Order Service says Yes; Inventory Service crashes = No vote
- 2PC aborts — Order Service rolls back. Clean, but requires 2PC infrastructure across both DBs
- Risk: if coordinator also fails at this moment, Order Service lock is held indefinitely

With a Saga:

- Order Service creates order (local commit). Publishes 'Order Created' event
- Inventory Service processes event, tries to reserve — crashes
- Compensating transaction: delete the order OR mark it as 'failed' — no distributed locking needed
- Much simpler failure recovery; no blocking of other services

Implementing Sagas: Two Approaches



Orchestrated Saga

Command & Control

- A central Orchestrator controls the entire flow
- Orchestrator calls each service in sequence
- Services don't know about each other
- Easy to visualize — process lives in one place
- Uses request-response communication
- Relies on a central Order Processor / Saga Orchestrator service

Choreographed Saga

Trust But Verify

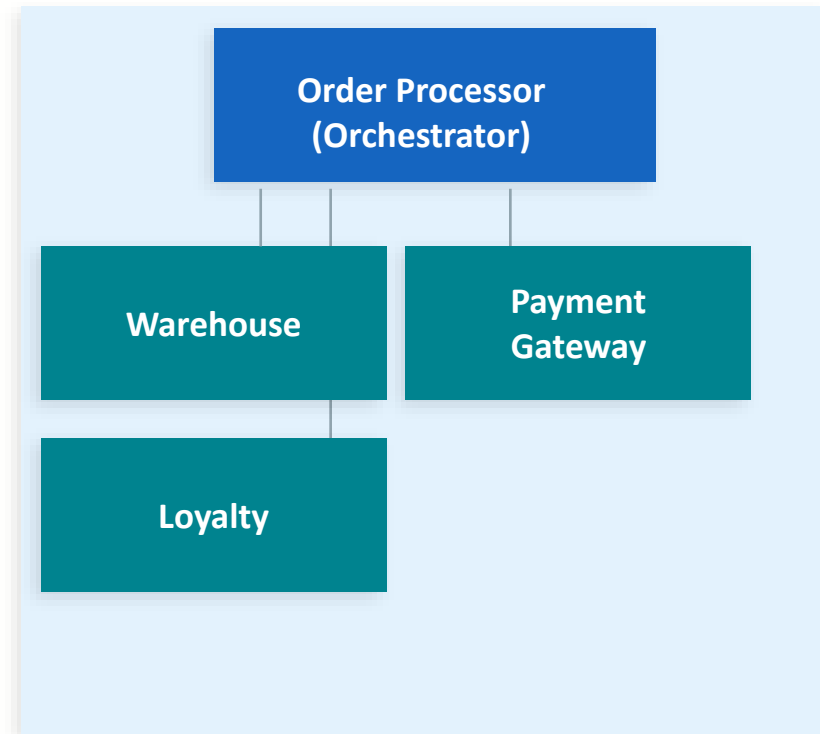
- No central controller — services react to events
- Each service emits events after completing its step
- Next service listens for the event and acts
- Highly decoupled — services don't know each other
- Relies on a message broker (RabbitMQ, Kafka)
- Tracking overall state requires correlation IDs

Orchestrated Sagas: Deep-Dive



The Order Processor as the central brain

- The orchestrator knows the full process flow and calls services in order
- Services are kept thin — they just respond to requests, don't manage flow
- If a service call fails, the orchestrator decides what to do (retry, compensate, alert)
- **Risk: logic creep — business logic can migrate into the orchestrator**
- Best practice: have DIFFERENT orchestrators per flow (Order, Returns, Goods Receiving...)
- Orchestrators make heavy use of request-response (REST/gRPC calls)



Orchestrated Saga: Python Implementation



FastAPI-based orchestrator coordinating multiple services

```
1 # order_orchestrator.py (FastAPI)
2 import httpx
3 from fastapi import FastAPI, HTTPException
4
5 app = FastAPI()
6
7 WAREHOUSE_URL = "http://warehouse-service"
8 PAYMENT_URL = "http://payment-service"
9 LOYALTY_URL = "http://loyalty-service"
10
11 @app.post("/orders/{order_id}/fulfill")
12 async def fulfill_order(order_id: str):
13     reserved = charged = None
14     async with httpx.AsyncClient() as client:
15         try:
16             # Step 1: Reserve stock
17             r = await client.post(
18                 f"{WAREHOUSE_URL}/reserve",
19                 json={"order_id": order_id})
20             r.raise_for_status()
21             reserved = r.json()["reservation_id"]
22
23             # Step 2: Charge payment
24             r = await client.post(
25                 f"{PAYMENT_URL}/charge",
26                 json={"order_id": order_id})
27             r.raise_for_status()
28             charged = r.json()["charge_id"]
29
30             # Step 3: Award loyalty points
31             await client.post(
32                 f"{LOYALTY_URL}/award",
33                 json={"order_id": order_id})
34
35             return {"status": "fulfilled"}
36
37 except Exception as e:
38     # Compensate in reverse order
39     if charged:
40         await client.post(
41             f"{PAYMENT_URL}/refund",
42             json={"charge_id": charged})
43     if reserved:
44         await client.post(
45             f"{WAREHOUSE_URL}/cancel",
46             json={"reservation_id": reserved})
47     raise HTTPException(500, f"Saga failed: {e}")
```

Persisting Saga State for Crash Recovery



The orchestrator must survive restarts



```
1 # models.py - Saga state table
2 from sqlalchemy import Column, String, Enum, JSON
3 import enum
4
5 class SagaStatus(enum.Enum):
6     STARTED = "STARTED"
7     STOCK_RESERVED = "STOCK_RESERVED"
8     PAYMENT_CHARGED = "PAYMENT_CHARGED"
9     LOYALTY_AWARDED = "LOYALTY_AWARDED"
10    COMPLETED = "COMPLETED"
11    COMPENSATING = "COMPENSATING"
12    FAILED = "FAILED"
13
14 class OrderSaga(Base):
15     __tablename__ = "order_sagas"
16     order_id = Column(String, primary_key=True)
17     status = Column(Enum(SagaStatus))
18     reservation_id = Column(String, nullable=True)
19     charge_id = Column(String, nullable=True)
20     metadata = Column(JSON, default={})
21
```



```
1 # --- In the orchestrator ---
2 async def fulfill_order(order_id: str, db: Session):
3     saga = OrderSaga(order_id=order_id,
4                       status=SagaStatus.STARTED)
5     db.add(saga); db.commit()
6
7     # Step 1
8     reservation_id = await reserve_stock(order_id)
9     saga.status = SagaStatus.STOCK_RESERVED
10    saga.reservation_id = reservation_id
11    db.commit() # Persist before proceeding!
```

Each status transition = a DB commit = a recovery checkpoint

Choreographed Sagas: Events Instead of Commands



No central controller — services react to events

- **Events replace direct calls — services don't know about each other**
- The Warehouse listens for 'Order Placed' → reserves stock → emits 'Stock Reserved'
- Payment Gateway listens for 'Stock Reserved' → charges → emits 'Payment Taken'
- Loyalty listens for 'Payment Taken' → awards points → emits 'Points Awarded'
- Each service is autonomous; the process emerges from their collective behavior



Choreographed Saga: RabbitMQ Implementation

```
1 import pika, json
2
3 def handle_order_placed(ch, method, props, body):
4     event = json.loads(body)
5     order_id = event["order_id"]
6     item_id = event["item_id"]
7
8     try:
9         reservation_id = reserve_stock(order_id, item_id)
10        # Emit next event
11        publish_event("stock.reserved", {
12            "order_id": order_id,
13            "reservation_id": reservation_id,
14            "correlation_id": event["correlation_id"]
15        })
16        ch.basic_ack(delivery_tag=method.delivery_tag)
17    except StockUnavailableError:
18        # Emit failure event
19        publish_event("order.cancelled", {
20            "order_id": order_id,
21            "reason": "stock_unavailable",
22            "correlation_id": event["correlation_id"]
23        })
24        ch.basic_ack(delivery_tag=method.delivery_tag)
25
26 def publish_event(routing_key, payload):
27     channel.basic_publish(
28         exchange="orders",
29         routing_key=routing_key,
30         body=json.dumps(payload),
31         properties=pika.BasicProperties(
32             delivery_mode=2 # persistent message
33         )
34     )
```

- Each service subscribes to specific event types
- Publish next event on success, failure event on error
- **correlation_id** links all events in the same saga
- Message durability: `delivery_mode=2` persists to disk
- `basic_ack` tells the broker the message was processed
- Unacknowledged messages are redelivered on restart

Correlation IDs: Tracking Saga State in Choreography



Essential for observability and recovery

- A Correlation ID is a unique identifier assigned when the saga begins
- Every event fired as part of the saga carries this ID
- Every service log entry includes this ID
- Enables reconstruction of the full saga history from distributed logs
- A 'Saga State Tracker' service can consume all events and build a real-time view
- **Without correlation IDs, debugging a choreographed saga is nearly impossible**



```
1  import uuid
2
3  correlation_id = str(uuid.uuid4())
4
5  # Include in EVERY event
6  event = {
7      "order_id": order_id,
8      "item_id": item_id,
9      "correlation_id": correlation_id,
10     "timestamp": datetime.utcnow().isoformat()
11 }
12
13 # In EVERY service handler, extract and forward
14 correlation_id = incoming_event["correlation_id"]
15
16 # In EVERY log line
17 logger.info("Reserving stock",
18     extra={
19         "correlation_id": correlation_id,
20         "order_id": order_id,
21         "service": "warehouse"
22     })
```

Orchestration vs Choreography



Criteria	Orchestration	Choreography
Coupling	Higher (orchestrator knows everyone)	Lower (services only know events)
Visibility	Easy — process in one place	Hard — need correlation + tracker
Team ownership	Best for one team owning whole saga	Best for multiple teams
Testing	Easier (single entry point)	Harder (distributed behavior)
Failure recovery	Orchestrator decides	Each service handles independently
Scalability	Orchestrator is a bottleneck	Each service scales independently
Logic creep risk	High — logic migrates to orchestrator	Low — logic stays in services

Design Challenge



Scenario: Ride-Sharing App

You are building a ride booking flow across 4 microservices:

- Booking Service — creates a ride booking record
- Driver Service — assigns a nearby driver
- Billing Service — pre-authorizes the customer's card
- Notification Service — sends confirmation to customer and driver

Your tasks:

- Decide: orchestrated or choreographed saga? Justify your choice.
- List the compensating transaction for each forward step
- Identify which step is most likely to fail and should go earliest
- What happens if notification fails? Should we roll back the whole booking?

Design Challenge: Suggested Solution



Step	Forward Action	Compensating Action
1	Billing: Pre-authorize card (most likely to fail — do first!)	Release pre-auth
2	Driver: Assign driver	Unassign driver / put back in pool
3	Booking: Create booking record	Delete/cancel booking record
4	Notification: Send confirmations	Send cancellation notification

- Architecture choice: If one team owns all 4 services → orchestration (simpler). If separate teams → choreography (more autonomous).
- Notification failure: Use fail-forward (retry). Don't roll back a confirmed ride just because an SMS failed — that's a technical, not business failure.
- Billing goes first because card declines are the most common failure — no point assigning a driver if payment won't work.

Monitoring & Troubleshooting Sagas



Observability is non-negotiable in distributed workflows

- **Challenge: when a saga fails, the cause may span 3+ services and 10+ events**
- Without correlation IDs + structured logging, debugging is a nightmare
- Build a saga state tracker — a service that consumes all events and builds a timeline view
- Alert on: saga stuck in intermediate state for > N minutes, compensation rate > X%, total failure rate



```
1 # saga_tracker.py - listens to all events
2 from kafka import KafkaConsumer
3 import json
4
5 consumer = KafkaConsumer(
6     "orders.*", # ALL order-related topics
7     bootstrap_servers="kafka:9092",
8     group_id="saga-tracker"
9 )
10
11 def update_saga_state(db, event):
12     saga = db.query(SagaState)\
13         .filter_by(
14             correlation_id=event["correlation_id"]
15         ).first()
16     if not saga:
17         saga = SagaState(
18             correlation_id=event["correlation_id"])
19         db.add(saga)
20
21     saga.last_event = event["type"]
22     saga.last_updated = datetime.utcnow()
23     saga.history = saga.history + [event]
24     db.commit()
```

Common Saga Mistakes to Avoid



1. Non-idempotent handlers

✓ Always deduplicate using event_id before processing

2. Forgetting to define compensations upfront

✓ Design compensations for every step BEFORE implementation

3. Not persisting saga state (orchestration)

✓ Commit saga state to DB before calling each next service

4. Confusion between business and technical failures

✓ Technical failures → retry/circuit breaker. Business failures → compensate

5. Giant orchestrator absorbing all business logic

✓ Keep logic in services; orchestrator only controls sequence

6. No correlation IDs (choreography)

✓ Correlation ID is mandatory for every saga event



Appendix A

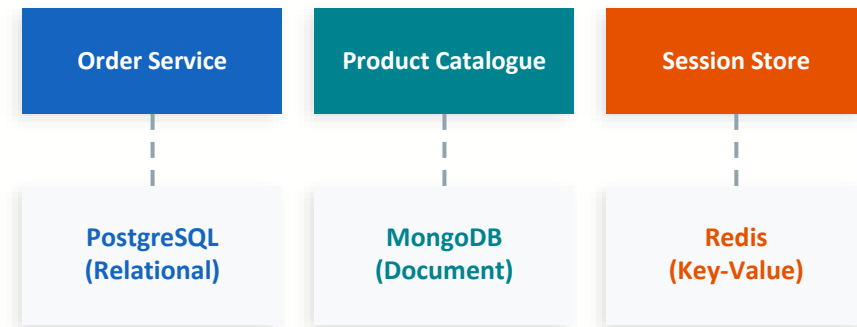
SQL vs. NoSQL in Microservices

Choosing the right data store for each service

The Service-per-Database Pattern



- **A foundational microservices principle: each service manages its own private data store**
- No shared databases between services — coupling at the data layer undermines service autonomy
- Consequence: different services may, and often should, use different database technologies
- This is called polyglot persistence — choosing the right storage model per business need
- The question is no longer 'which database?', but 'which database for which service?'



Polyglot persistence

Implication for transactions: because each service owns its own database, cross-service operations cannot use a single ACID transaction — this is the direct motivation for the Saga pattern.

Relational Databases (SQL)



- **Strengths**
 - ACID transactions: full atomicity, consistency, isolation, durability
 - Powerful query language (SQL): joins, aggregations, sub-queries
 - Strong schema enforcement: data integrity guaranteed at the DB layer
 - Mature ecosystem: excellent tooling, ORM support, replication, backups
 - Best for: financial records, order management, user accounts — any domain with strict consistency requirements
- **Trade-offs**
 - Schema changes (migrations) require careful coordination across services
 - Horizontal scaling is more complex than NoSQL alternatives
 - Fixed schema reduces flexibility when data structures evolve frequently

Use relational databases when your service requires strong consistency, complex queries, or well-defined relational data.

NoSQL Databases: Why Do They Exist?



Relational databases are excellent — but they were designed for structured, predictable data. As the internet scaled to billions of users, new problems appeared that SQL alone struggled to handle.



Unpredictable structure

A product catalogue might have 5 fields for a t-shirt, and 40 fields for a laptop. A fixed SQL schema wastes space or requires constant ALTER TABLE migrations.



Extreme read speed

A login system checking a session token 10,000 times per second doesn't need complex joins — it just needs one key, one value, instantly.



Massive write volume

An IoT sensor logging temperature every second from 1 million devices generates billions of rows per day. Relational DBs struggle to keep up at this scale.

NoSQL ("Not Only SQL") is an umbrella term for databases that store data differently — using documents, key-value pairs, wide columns, or graphs — each designed to solve one of these specific problems efficiently.

Four Types of NoSQL Database



Document Store

MongoDB, CouchDB

Stores data as flexible JSON-like documents. Each document is self-contained and can have a completely different structure — no rigid schema required.

✓ **Best for:** Product catalogues, user profiles, blog posts — anywhere data shape varies.

✗ **Avoid when:** Complex relationships between many different types of data.

Column-Family Store

Apache Cassandra, HBase

Groups related columns together in 'families'. Unlike rows in SQL, each row can have billions of columns — optimised for massive write throughput and time-series data.

✓ **Best for:** Event logs, sensor readings, activity feeds, analytics at scale.

✗ **Avoid when:** Ad-hoc queries with many filters — you must know your query patterns upfront.

Key-Value Store

Redis, DynamoDB

The simplest NoSQL model — a giant dictionary. You store a value under a unique key and retrieve it instantly. Extremely fast because there's no query logic.

✓ **Best for:** User sessions, caches, shopping carts, feature flags, rate limiting.

✗ **Avoid when:** Searching by anything other than the exact key.

Graph Database

Neo4j, Amazon Neptune

Models data as nodes (entities) and edges (relationships). Traversing connections is native and fast — there are no expensive JOIN operations like in SQL.

✓ **Best for:** Social networks, recommendation engines, fraud detection, knowledge graphs.

✗ **Avoid when:** Simple CRUD operations — adds unnecessary complexity for basic data.

What NoSQL Gives Up to Gain Speed and Scale



⚠ No ACID guarantees (usually)

● SQL: Bank transfer — debit AND credit happen together or not at all. Guaranteed.

● NoSQL: A document update might succeed while a related update fails. The app must handle this.

→ **Most NoSQL stores use eventual consistency — the data will be correct eventually, but for a short moment you might read stale data.**

🔗 No joins — data must be duplicated

● SQL: One Orders table, one Customers table — join them on demand.

● NoSQL: You embed the customer's name inside every order document. If the name changes, you update it everywhere.

→ **Denormalisation trades storage space for read speed. You optimise for how you read, not how you store.**

📋 You must know your queries upfront

● SQL: Write any query later — SELECT, GROUP BY, WHERE, any column, any order.

● NoSQL: Many stores (e.g. Cassandra) require you to design your tables around the queries you plan to run. Ad-hoc queries are slow or impossible.

→ **Design your NoSQL schema around your access patterns, not around your data model.**

Selecting the Appropriate Database per Service



Service / Domain	Recommended Database	Primary Reason
Order / Payment	PostgreSQL / MySQL	ACID transactions
Product Catalogue	MongoDB / Elasticsearch	Flexible schemas, full-text search
User Sessions / Cache	Redis / Memcached	Sub-millisecond key-value access
Activity Feed / Events	Apache Cassandra	High write throughput, time-ordered reads
Social Graph / Recommendations	Neo4j / Amazon Neptune	Relationship traversal queries
Search & Discovery	Elasticsearch / OpenSearch	Inverted index, relevance ranking
Workflow / Saga State	PostgreSQL or DynamoDB	ACID or strong consistency for saga checkpointing

Design principle: choose the database that best fits the service's access patterns and consistency requirements — not the one your organisation is already most familiar with.



Architecture Decision Guide

Putting it all together — how to decide in practice

Session Summary: Key Takeaways



1

ACID transactions guarantee atomicity within a single service's DB — but cannot span multiple services

2

Two-Phase Commit seems like the solution but introduces long locks, failure modes, and scalability problems

3

Sagas model multi-service workflows as sequences of local transactions + compensating actions

4

Orchestrated sagas use a central controller (great for one team). Choreographed sagas use events (great for multiple teams)

5

Always define compensating transactions before implementing forward actions

6

Correlation IDs, idempotent handlers, and saga state persistence are non-negotiable for production sagas

Quick Decision Reference



Situation	Recommendation	Pattern
Single service, multi-table update	Use local ACID transaction	✓ ACID
2+ services, need atomicity	Consider keeping data in one service, OR...	Saga
one team, visibility needed	Orchestrated saga (central controller)	Orchestration
multiple teams	Choreographed saga (event-driven)	Choreography
Cross-service, distributed lock needed	Redesign — avoid 2PC, use Saga	⚠ Avoid 2PC
Transient failures (network, timeouts)	Retry with exponential backoff + circuit breaker	Retry/CB
Business rule violation	Compensating transaction	Compensate

Saga Architecture Decision Framework



Q: Do I even need a saga?

→ Only if the operation spans 2+ services/databases.
Single service → use local ACID transactions.

Q: How many teams own the services?

→ One team → orchestration OK. Multiple teams → choreography preferred.

Q: How easy is compensation?

→ Hard-to-compensate steps should go last. Move most-likely-to-fail steps first.

Q: Can I keep this data in one service?

→ If state changes need true atomicity, keep them in one service. Decompose later.

Q: What kind of failures do I expect?

→ Business failures → compensate. Technical failures → retry with circuit breaker.

When Sagas Are Overkill



- **Sagas add significant complexity — use them only when genuinely needed**
- You do NOT need a saga when:
 - The entire operation fits in one service's database
 - Eventual consistency is unacceptable and you need strong consistency (reconsider the architecture)
 - The 'workflow' is just a synchronous API call with a retry
- **Consider alternatives:**
 - Defer decomposition: keep in a monolith until you understand the domain better
 - Two-step HTTP with compensation: simple for 2 services, no saga infrastructure

Complexity spectrum

Single service, single DB

Local transaction

2 services, 2 DBs

Simple compensation

3-5 services

Orchestrated saga

5+ services, multi-team

Choreographed saga

The Big Picture: Why This Matters



- Microservices trade the simplicity of local transactions for the flexibility of distributed systems
- **The saga pattern is how you maintain business integrity without distributed locking**
- Explicitly modeled business processes → better observability, easier onboarding, fewer surprises
- Every complex system you will build involves multi-service workflows — this pattern is universally applicable
- The engineering discipline of distributed systems is fundamentally about making peace with failure — and designing for it gracefully



Making the core business processes of your system a first-class concept will have a host of advantages.

— Sam Newman
Building Microservices

Textbooks

- 
- Sam Newman, "Building Microservices: Designing Fine-Grained Systems," O'Reilly Media, 2021.



Thank You!

Questions?